

CUSTOMER CHURN PREDICTION

Using Machine Learning & Exploratory Data Analysis

Internship	Codec Technologies – 1-Month Data Analytics Internship
Project	Customer Churn Prediction
Prepared by	Wasif Sayyed
Dataset	IBM Telco Customer Churn Sample Dataset
Tools	Python, Pandas, NumPy, Matplotlib, Scikit-learn
Models	Logistic Regression, Random Forest, Gradient Boosting

Submission note: This report is structured for an internship portfolio. The accompanying Python script performs the complete data cleaning, EDA, model training and evaluation workflow. Exact evaluation scores are generated by executing the script against the original CSV dataset, rather than being invented in this report.

1. Project Overview

Customer churn means a customer stops using a company's service. In telecom, identifying customers who are likely to leave can help the business take preventive retention actions. This project builds a supervised machine-learning classification pipeline that predicts whether a customer is likely to churn.

Goal: Predict which customers are likely to stop using a telecom service.

Business objective: identify high-risk customers early and support targeted retention campaigns.

2. Dataset Description

The project uses the IBM Telco Customer Churn sample dataset. It contains 7,043 customer records and 21 columns covering demographics, tenure, subscribed services, contract information, billing and the churn outcome. The target variable is **Churn**, indicating whether the customer left within the observed period.

Category	Examples
Customer profile	gender, SeniorCitizen, Partner, Dependents
Tenure & services	tenure, PhoneService, MultipleLines, InternetService
Support/services	OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport
Entertainment	StreamingTV, StreamingMovies
Account	Contract, PaperlessBilling, PaymentMethod
Financial	MonthlyCharges, TotalCharges
Target	Churn (Yes / No)

Source: IBM Telco Customer Churn sample dataset. The public dataset is also mirrored on Kaggle and other repositories. The original IBM repository is archived.

3. Project Methodology

1	Load dataset	Read the CSV file and inspect shape, columns and data types.
2	Clean data	Convert TotalCharges to numeric, handle missing values and remove the identifier.
3	EDA	Study churn distribution and churn patterns across contract and service variables.
4	Preprocess	Impute missing values, scale numerical variables and one-hot encode categorical variables.
5	Train models	Train Logistic Regression, Random Forest and Gradient Boosting classifiers.
6	Evaluate	Compare accuracy, precision, recall, F1-score and ROC-AUC.
7	Risk scoring	Convert predicted churn probabilities into Low, Medium and High risk groups.

4. Exploratory Data Analysis (EDA)

The EDA stage checks the target distribution and looks for relationships that may help explain customer churn. Useful comparisons include churn by contract type, internet service, tenure and payment method.

Expected business patterns to investigate

- Month-to-month customers generally show substantially higher churn than customers on longer contracts.
- Customers with shorter tenure are an important high-risk segment.

- Electronic-check users can show higher churn than customers using automatic payment methods.
- Internet-service type and lack of support/security services can be useful predictive variables.

Important: These are analytical hypotheses/patterns to verify from the supplied dataset. The Python script prints the exact group-wise churn rates so the final GitHub notebook can show the actual values from the dataset.

5. Data Preprocessing

The customerID field is removed because it is an identifier rather than a meaningful predictive feature. TotalCharges is converted from text to numeric; blank values become missing values and are imputed during the modeling pipeline. Categorical features are one-hot encoded, while numerical features are median-imputed and standardized.

Train/Test Strategy

The data is split into 80% training and 20% testing using stratification so the churn class proportion is kept approximately consistent in both sets. random_state=42 is used for reproducibility.

6. Machine Learning Models

Model	Why it is used
Logistic Regression	Strong baseline, fast and interpretable for binary classification.
Random Forest	Captures nonlinear relationships and interactions between customer attributes.
Gradient Boosting	Builds an ensemble of sequential trees and can model complex patterns.

7. Evaluation Metrics

Metric	Meaning
Accuracy	Percentage of all test predictions that are correct.
Precision	Among customers predicted to churn, the proportion who actually churned.
Recall	Among actual churners, the proportion correctly identified.
F1-score	Harmonic mean of precision and recall.
ROC-AUC	Measures how well the model ranks churners above non-churners across thresholds.

8. Reproducible Results Table

The exact scores below are intentionally generated by the accompanying script. This avoids presenting unverified or fabricated metrics. After execution, the script prints a model-comparison table containing Accuracy, Precision, Recall, F1 and ROC-AUC and saves it as **model_comparison.csv**.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	Run script	Run script	Run script	Run script	Run script
Random Forest	Run script	Run script	Run script	Run script	Run script
Gradient Boosting	Run script	Run script	Run script	Run script	Run script

9. Complete Python Implementation

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
df = df.drop_duplicates()
df = df.drop(columns=["customerID"])
df["Churn"] = df["Churn"].map({"Yes": 1, "No": 0})

X = df.drop(columns=["Churn"])
y = df["Churn"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

categorical_cols = X.select_dtypes(include=["object"]).columns.tolist()
numeric_cols = X.select_dtypes(exclude=["object"]).columns.tolist()

preprocessor = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]), numeric_cols),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))
    ]), categorical_cols)
])

models = {
    "Logistic Regression": LogisticRegression(max_iter=2000, class_weight="balanced"),
    "Random Forest": RandomForestClassifier(
        n_estimators=400, random_state=42, class_weight="balanced"
    ),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42)
}

results = []

for name, model in models.items():
    pipe = Pipeline([
        ("preprocessor", preprocessor),
        ("model", model)
    ])
    pipe.fit(X_train, y_train)

    pred = pipe.predict(X_test)
    prob = pipe.predict_proba(X_test)[:, 1]

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, pred),
        "Precision": precision_score(y_test, pred),
        "Recall": recall_score(y_test, pred),
        "F1": f1_score(y_test, pred),
        "ROC-AUC": roc_auc_score(y_test, prob)
    })

results_df = pd.DataFrame(results).sort_values("ROC-AUC", ascending=False)
print(results_df.round(4))
results_df.to_csv("model_comparison.csv", index=False)

# Churn patterns
print(df.groupby("Contract")["Churn"].mean().sort_values(ascending=False))
print(df.groupby("InternetService")["Churn"].mean().sort_values(ascending=False))

# ROC curve
from sklearn.metrics import RocCurveDisplay
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))
for name, model in models.items():
    pipe = Pipeline([("preprocessor", preprocessor), ("model", model)])
    pipe.fit(X_train, y_train)
    RocCurveDisplay.from_predictions(
        y_test, pipe.predict_proba(X_test)[:, 1], name=name
    )

plt.title("ROC Curves - Customer Churn Models")

```

```
plt.tight_layout()  
plt.show()
```

10. Business Interpretation & Retention Strategy

- **High-risk segment:** customers with high predicted churn probability can be prioritized for retention outreach.
- **Contract strategy:** month-to-month customers can be offered suitable incentives for longer-term plans.
- **Early-tenure support:** new customers can receive onboarding and technical support during their first months.
- **Service experience:** customers without support/security services can be examined for service-quality issues.
- **Payment friction:** customers using manual payment methods can be encouraged toward convenient automatic payment options.

11. Limitations

- The IBM dataset is a sample dataset for a fictional telecom company; it is not a production customer database.
- The data is cross-sectional, so this project should be described as churn classification rather than a time-series forecast.
- Predictions identify statistical risk and do not prove that a particular retention action will prevent churn.
- Model performance can change when the model is trained on a different company's customer population.

12. Conclusion

This project demonstrates an end-to-end customer churn analytics workflow: data cleaning, exploratory analysis, preprocessing, classification, evaluation and customer risk scoring. The approach gives a telecom business a practical framework for identifying customers who may need proactive retention attention. The accompanying script is reproducible and produces the final model-comparison metrics from the original dataset.

13. GitHub Submission Checklist

- WA_Fn-UseC_-Telco-Customer-Churn.csv (or a dataset download instruction rather than redistributing the file)
- customer_churn_prediction.py
- Customer_Churn_Prediction_Internship_Project.pdf
- model_comparison.csv (generated after running the script)
- README.md with project objective, dataset source, setup, methodology and results

14. Dataset Reference

IBM, Telco Customer Churn sample dataset. Public repository:
<https://github.com/IBM/telco-customer-churn-on-icp4d/blob/master/data/Telco-Customer-Churn.csv>

Prepared for Codec Technologies Data Analytics Internship. For educational/portfolio use.